IIT (ISM) Dhanbad

Department of Computer Science and Engineering

xv6-riscv Operating System Customization Project

Course: Operating Systems Lab (NSCS210)

1. **Instructor:** Jaishree Mayank Mam

Team Members

Prathamesh Vispute

Pranjul

Prathyush P

Punit

Poka Sireesha

Prabir Soren

GitHub: https://github.com/PrathamandX/G15_Project1_xv6CustomizeSystemCalls

Introduction

The xv6 operating system is a simple, Unix-like teaching OS that helps in understanding core operating system concepts. In this project, we extend the xv6-riscv kernel by implementing new system calls and modifying existing ones.

The main objective of this project is to gain hands-on experience with kernel-level programming, process management, inter-process communication (IPC), scheduling, threads, and synchronization mechanisms.

By modifying kernel files and adding user-space test programs, we demonstrate how operating system functionalities can be extended in a modular and efficient way.

Objectives

- Understand System Call Flow in xv6:
  To study how system calls are invoked from user space and handled within the kernel.
- Implement New System Calls:
  To design and add new system calls for different operating system functionalities.
- Modify Kernel Behavior Safely:
  To extend kernel features without affecting existing system stability and functionality.

- Demonstrate Features Using User Programs:
  To validate implemented features through user-level test programs.
- Learn Synchronization and Scheduling Concepts:
  To understand process/thread coordination and CPU scheduling mechanisms in an operating system.

Implemented Features

Feature	Details
waitpid()	Waits for a specific child process.
Shared Memory IPC	Adds shmget(int key) and shmdt(int key).
Priority Scheduling	Adds process priority and setp(int pid, int priority).
ismyear()	Returns the constant value 1926.
Kernel Threads	Adds thread_create(), thread_join(), thread_exit(), and thread_id().
Synchronization	Adds mutex and semaphore system calls.

Key Files Modified

- kernel/proc.c, kernel/proc.h
  Modified to implement process management features such as scheduling, thread handling, and process control logic.
- kernel/syscall.c, kernel/syscall.h
  Updated to register newly added system calls and assign unique syscall numbers.
- kernel/sysproc.c
  Implemented the core logic of newly introduced system calls at the kernel level.
- kernel/vm.c
  Modified to support memory-related features, particularly shared memory implementation.
- user/user.h
  Updated to declare user-level interfaces for the new system calls.
- user/usys.pl
  Modified to generate system call stubs that connect user programs with kernel implementations.
- Makefile
  Updated to include new files and ensure proper compilation of added features and test programs.

Testing

Commands and representative outputs:

$ test_ismyear

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ test_ismyear
ismyear returned: 1926
```

$ test_priority

```
$ test_priority
Child with priority 100 finished
Child with priority 50 finished
Child with priority 10 finished
$
```

$ waitpidtest

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ waitpidtest
Parent waiting specifically for child pid = 6
waitpid returned pid = 6, status = 33
Now collecting remaining children using wait()
wait collected one child, status = 22
wait collected one child, status = 11
$
```

$ shmtest

```
wait collected one child, status = 22
wait collected one child, status = 11
$ shmtest
child read: hello from parent
parent sees after child: ack from child
$
```

$ test_thread

$

```
$ test_thread
creating 4 threads
sys_thread_create: fcn=0 arg=0 stack=13000
kthread_create: success tid=9
spawned thread 0 with tid=9
sys_thread_create: fcn=0 arg=1 stack=11ff0
kthread_create: success tid=10
spawned thread 1 with tid=10
sys_thread_create: fcn=0 arg=2 stack=10fe0
kthread_create: success tid=11
spawned thread 2 with tid=11
sys_thread_create: fcn=0 arg=3 stack=ffd0
kthread_create: success tid=12
spawned thread 3 with tid=12
thread 0 tid=9 finished
joined tid=9 exit=0
thread 1 tid=10 finished
joined tid=10 exit=1
thread 2 tid=11 finished
joined tthiread=11d 3 t id=e1x2 fit=i2
nished
joined tid=12 exit=3
final counter = 8000 expected = 8000
$
```

synctest

```
hart 1 starting
hart 2 starting
init: starting sh
$ synctest
child 4 increment 0
child 5 increment 0
child 4 increment 1
child 5 increment 1
child 4 increment 2
child 5 increment 2
child 4 increment 3
child 5 increment 3
child 4 increment 4
child 5 increment 4
child 4 increment 5
child 5 increment 5
child 5 increment 6
child 4 increment 6
child 5 increment 7
child 4 increment 7
```

Added System Calls

The following system calls were implemented to extend xv6 functionality across different operating system domains:

1. Process Control

- **waitpid(int pid, int status)*
  Waits for a specific child process identified by its PID and returns its exit status.

2. Inter-Process Communication (IPC)

- shmget(int key)
  Creates or accesses a shared memory segment.
- shmdt(int key)
  Detaches the shared memory segment from the process.

3. Scheduling

- setp(int pid, int priority)
  Sets the priority of a process.
- ismyear(void)
  Returns a constant value (1926) to verify system call functionality.

4. Threads

- **thread_create(void (*fcn)(void *), void *arg, void stack)*
  Creates a new thread.
- thread_join(int tid)
  Waits for a thread to finish execution.
- thread_exit(int status)
  Terminates the calling thread.
- thread_id(void)
  Returns the thread ID.

5. Synchronization

- mutex_init(int id)
  Initializes a mutex.
- mutex_acquire(int id)
  Acquires the mutex lock.
- mutex_release(int id)
  Releases the mutex lock.
- sem_init(int id, int value)
  Initializes a semaphore.
- sem_wait(int id)
  Decrements the semaphore and blocks if needed.
- sem_post(int id)
  Increments the semaphore and wakes waiting processes.

Team Contributions

- **Prathamesh Vispute:**
  Implemented the waitpid() system call for process control.
- **Pranjul:**
  Implemented shared memory system calls (shmget, shmdt).
- **Prabir Soren:**
  Implemented priority scheduling and the ismyear() system call.
- **Prathyush P:**
  Implemented synchronization mechanisms using mutexes and semaphores.
- **Punit:**
  Implemented kernel-level thread support (thread_create, thread_join, thread_exit).
- **Poka Sireesha:**
  Integrated all modules and prepared project documentation.

Challenges Faced

- Understanding xv6 Kernel Structure:
  The xv6 codebase is compact but complex, making it difficult initially to understand process flow and system call handling.
- System Call Implementation:
  Adding new system calls required careful modification of multiple files (syscall.c, sysproc.c, proc.c) and maintaining correct mappings.
- Debugging Kernel Code:
  Debugging errors inside the kernel was challenging due to limited debugging tools and required careful tracing using print statements.
- Synchronization Issues:
  Handling concurrent execution in threads and processes required proper use of mutexes and semaphores to avoid race conditions.
- Integration of Multiple Features:
  Ensuring that features like threads, shared memory, and scheduling worked together without conflicts was difficult.

Limitations

- Simplified Scheduling:
  The priority scheduling implementation does not include advanced features like starvation prevention or dynamic priority adjustment.
- Limited Shared Memory Design:
  Shared memory supports only fixed keys and a limited number of segments, making it less flexible compared to real systems.
- Basic Thread Implementation:
  Threads are implemented in a simplified manner and do not support advanced thread management features.
- Static Synchronization Objects:
  Mutexes and semaphores are managed using fixed IDs instead of dynamic allocation.

- Non-Deterministic Output:
  Due to concurrent execution, outputs (especially in threads and synchronization tests) may appear interleaved.

Conclusion

In this project, the xv6-riscv operating system was extended by implementing system calls for process control, IPC, scheduling, threads, and synchronization.

These features were integrated into the kernel while preserving existing functionality and were validated using user-level test programs.

Overall, the project provided practical insights into kernel-level programming and improved understanding of key OS concepts such as concurrency, synchronization, and resource management.